

SISTEMAS OPERACIONAIS

```
mkdir "Universidade Federal de Alfenas"  
touch "Universidade Federal de Alfenas/Turma SO 2025-02.txt"  
echo "Professor Romário Borges" > "Universidade Federal de Alfenas/Turma SO  
2025-02.txt"
```

Threads

```
cat aula01.txt
```

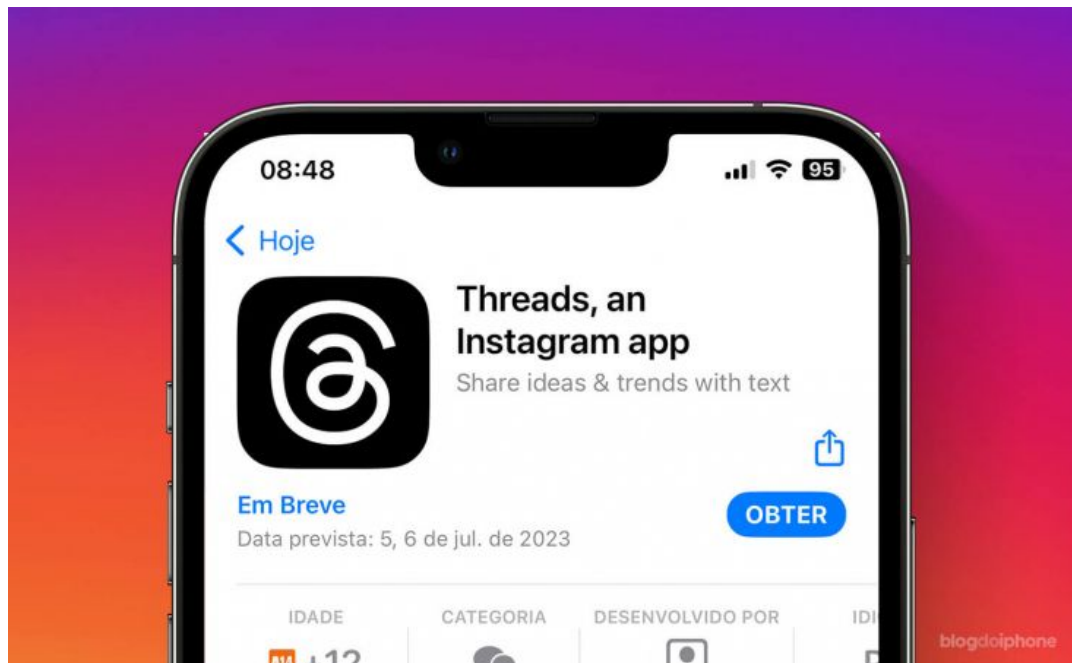
Na aula de Hoje, teremos:

- Conceito de Threads;
- As diferenças entre Threads e Processos;
- Os modelos de Threads de usuário e de Núcleo e suas relações;
- Modos de implementação de Threads (Explícito e implícito).

Recapitulando a aula anterior

- Conceito de Processo;
- Criação e término de um processo;
- Estados de um processo (pronto, executando, bloqueado);
- A hierarquia de processos (Unix e Windows);

Threads



Um pouco de história...

A necessidade de executar mais de uma tarefa

À medida em que as aplicações se tornavam mais complexas, mais tarefas por processos eram necessárias.

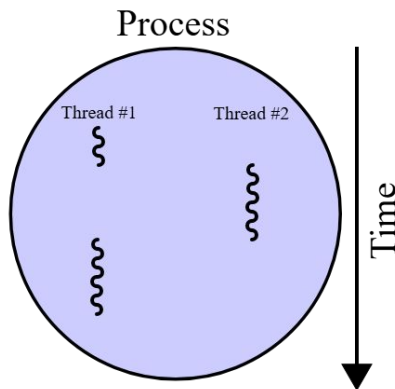
Origem de Threads

A necessidade de suportar mais de uma tarefa operando sobre os mesmos recursos, ou seja, dentro do mesmo processo.

Definição

Thread

É um fluxo de execução independente. Unidade básica de utilização da CPU. Unidade de execução dentro de um processo.



Threads

Analogia do que seria um processo e várias threads relacionadas.

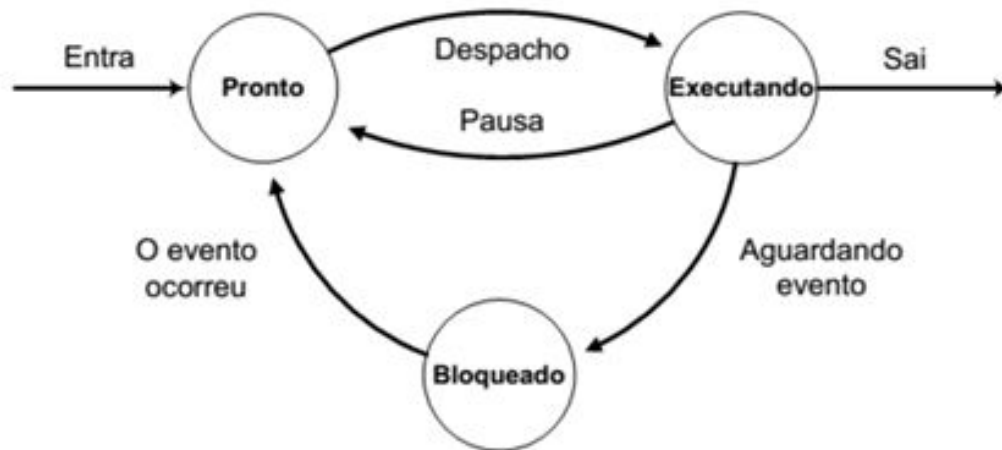


Característica

- Threads **compartilham as mesmas variáveis globais do processo**, arquivos abertos, processos filho;
- A sua “independência” reside apenas em suas variáveis locais e na sua própria pilha de execução.
- Cada thread **pode ter acesso a qualquer endereço de memória dentro do espaço de endereçamento do processo**.

Estados de um Thread

São os mesmos de um Processo.



Ciclo de vida de um Thread

- **Criação:** Quando ocorre a execução de múltiplos threads, os processos normalmente começam com apenas um dele, e cria novos a partir de procedimentos de uma biblioteca.
- **Término:** Depois disso o thread deixa de existir e não pertencerá mais a fila de threads escalonáveis. Libera totalmente o seu subespaço no espaço de endereçamento do processo.

Ciclo de vida de um Thread

- **Auto bloqueio:** O thread pode se bloquear para esperar outro thread terminar sua execução;
- **Voluntário:** Ou pode desistir voluntariamente da utilização da CPU, para deixar outro thread executar.

Característica

Um Thread é composto por:

- ID
- Contador de programas
- Conjunto de registradores
- Pilha

Recapitulando a aula passada...

Bloco de Controle de Processo

Gerenciamento de processo
Registros
Contador de programa
Palavra de estado do programa
Ponteiro da pilha
Estado do processo
Prioridade
Parâmetros de escalonamento
ID do processo
Processo pai
Grupo de processo
Sinais
Momento em que um processo foi iniciado
Tempo de CPU usado
Tempo de CPU do processo filho
Tempo do alarme seguinte

Processso X Thread

Processo x Thread

Processo

Definição: Programa em execução com recursos próprios;

Memória: próprio espaço;

Custo de criação: alto;

Recursos: possui pilha, variáveis globais, arquivos abertos e conexões independentes;

Segurança: maior isolamento e falha geralmente não afeta os demais;

Troca de contexto: Mais custosa;

Thread

Definição: Unidade de execução dentro de um processo;

Memória: compartilham espaço;

Custo de criação: baixo;

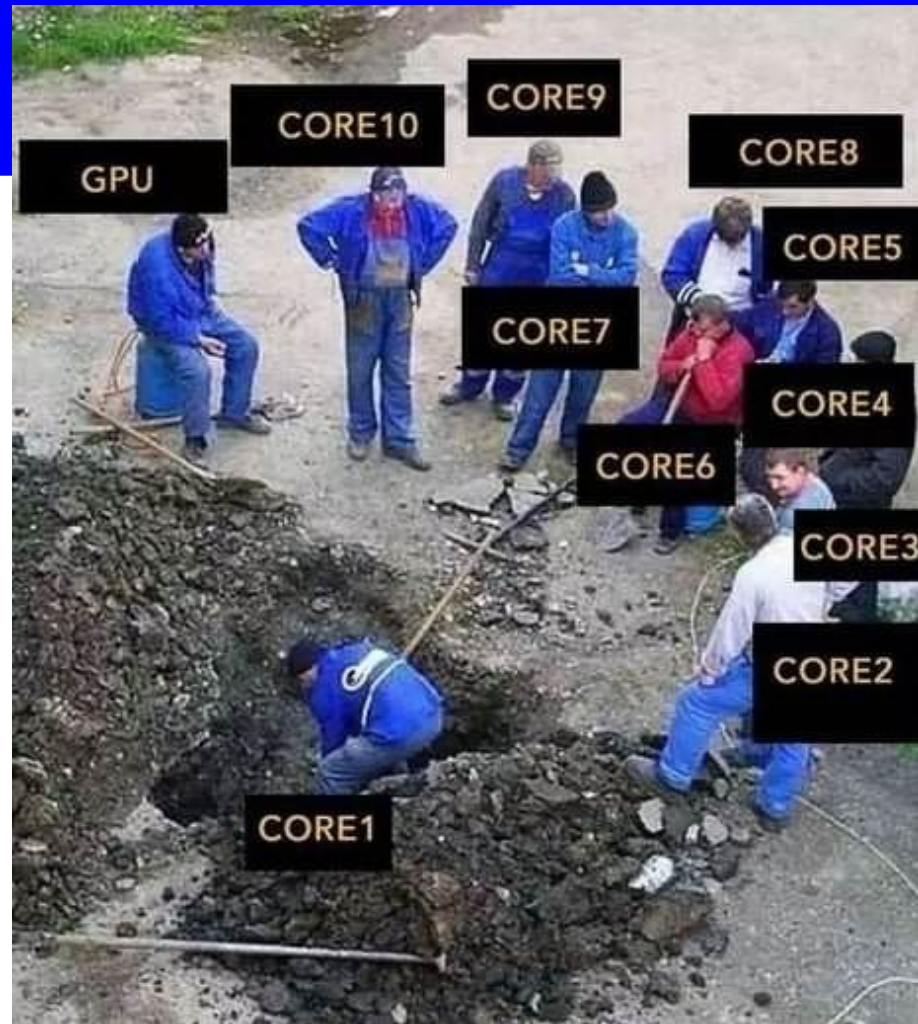
Recursos: Compartilham variáveis globais e arquivos do processo, mas cada um tem sua própria pilha;

Segurança: menor isolamento e falha pode afetar todo o processo;

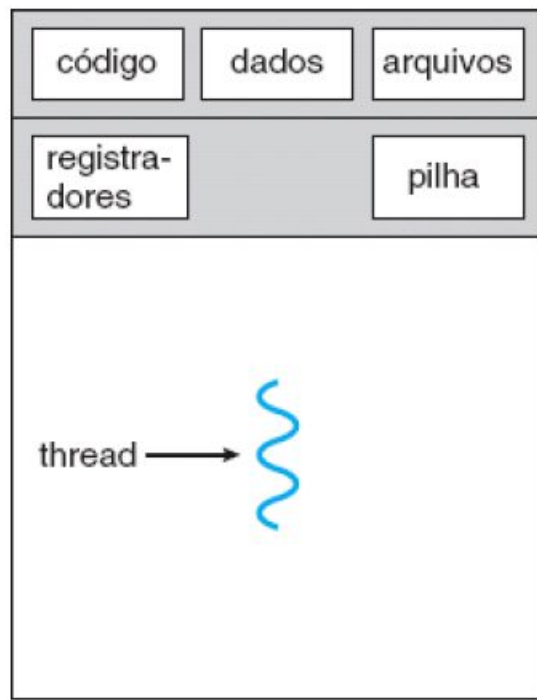
Troca de contexto: Muito mais leve;

MultiThread

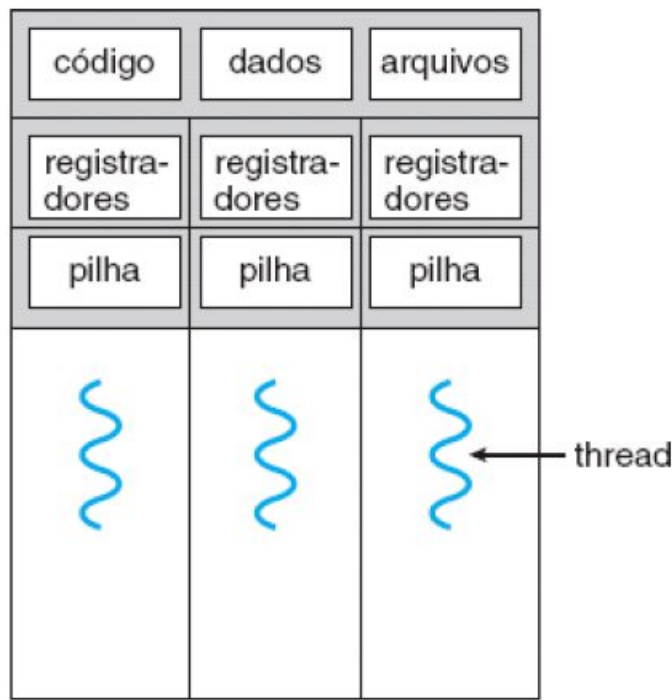
Trabalho Multithread:
“Dividir para conquistar”



MultiThread



processo com um único thread



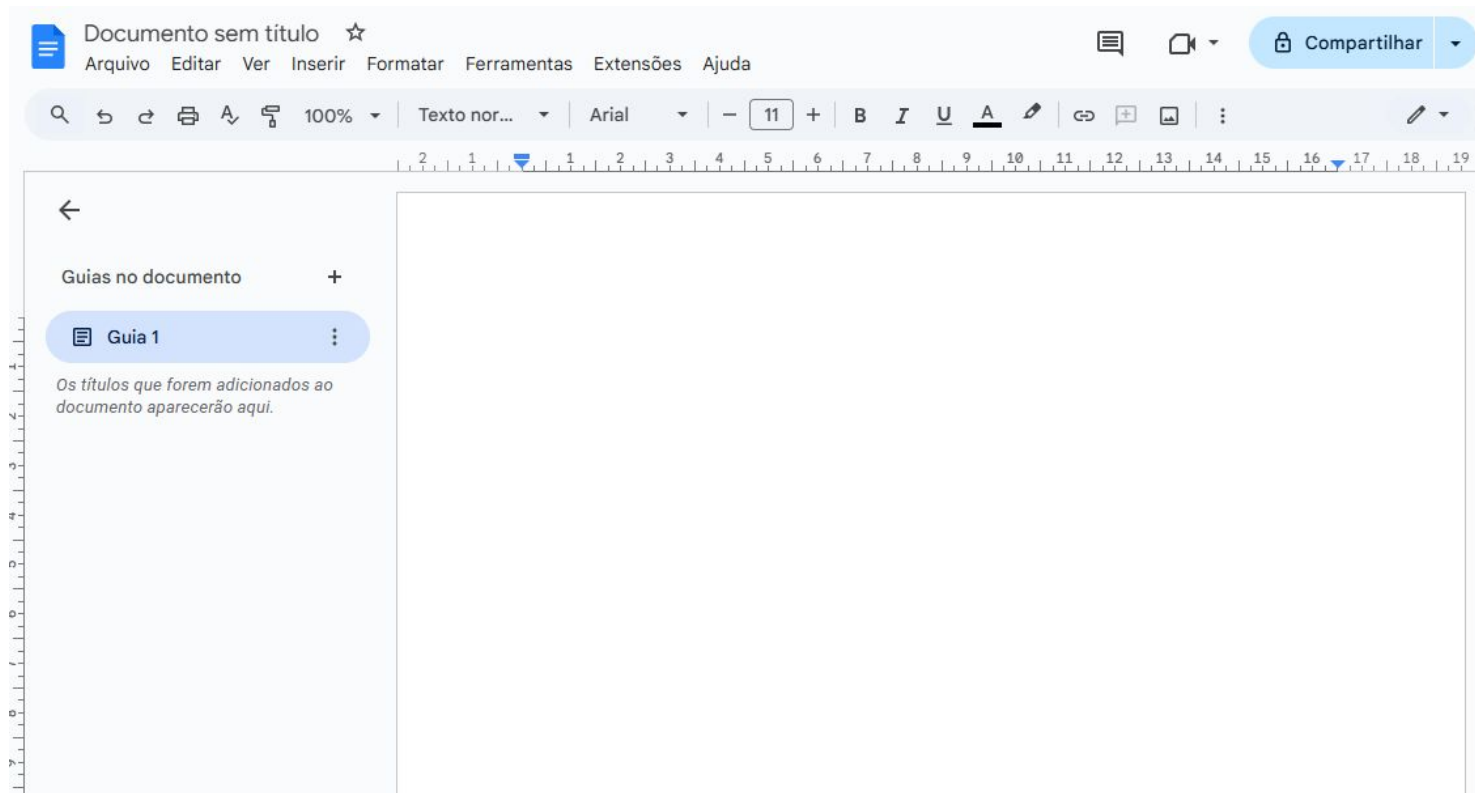
processo com múltiplos threads

Programação Multicore

Em resposta a necessidade de melhor desempenho de computação, múltiplos núcleos passaram a ser implementados no mesmo chip.

A programação multithread fornece mecanismos para uso mais eficiente desses múltiplos núcleos.

Exemplo de um editor de texto



Para refletir

No exemplo do editor de texto, o que podemos pensar sobre ter **N processos** executando, ao invés de ter **N threads** executando?

Portanto...

Benefícios de Multithread

- Capacidade de resposta;
- Compartilhamento de recursos;
- Economia;
- Escalabilidade



Modelos de implementação de Threads

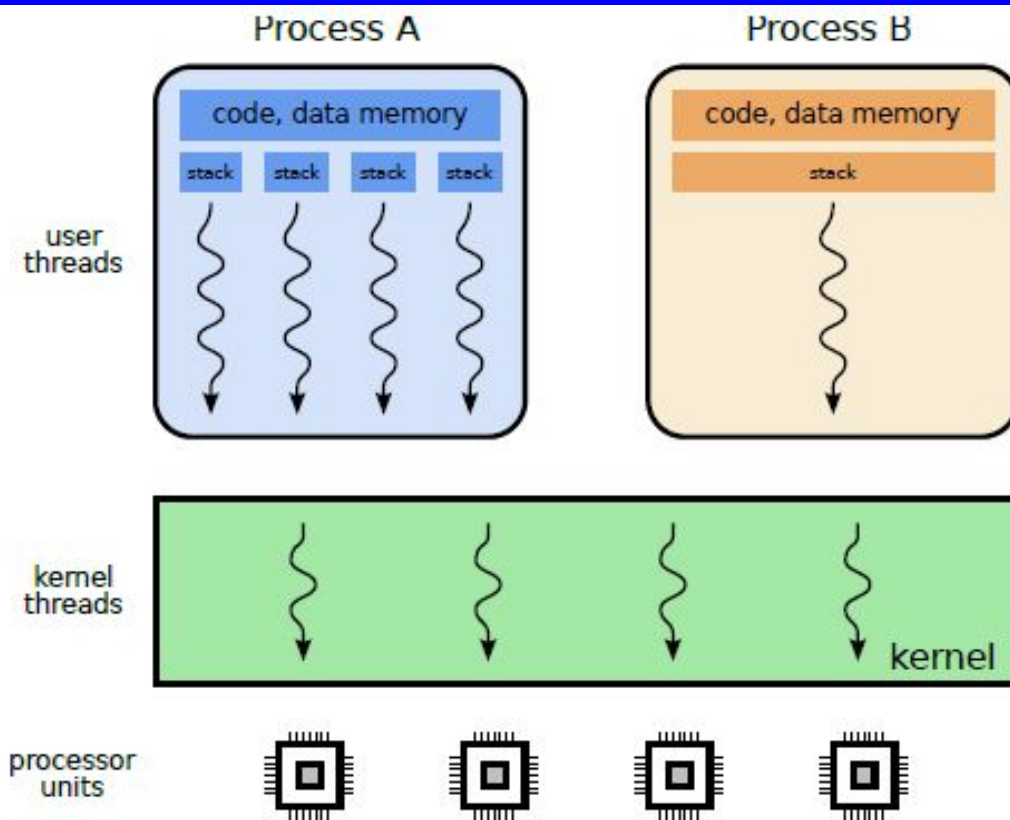


Figura 5.4: *Threads* de usuário e de núcleo.

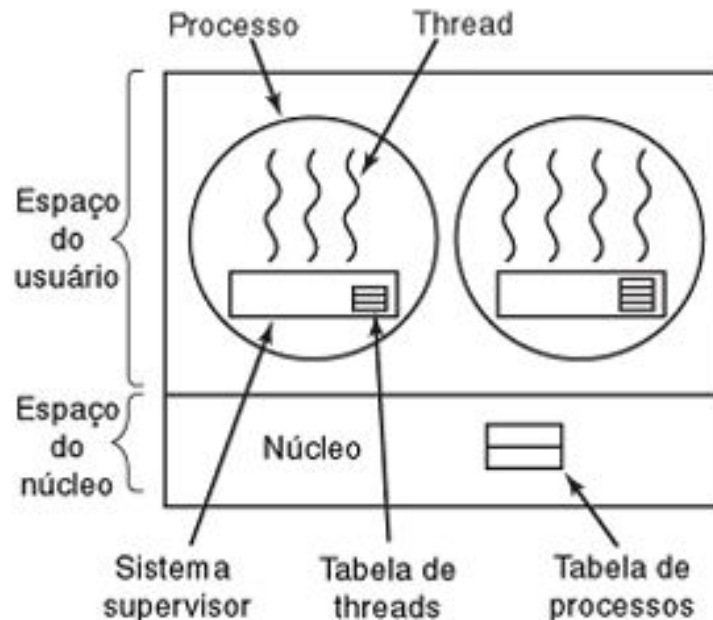
Threads no espaço de Usuário

Threads de usuário (a nível do programador)

São implementadas no espaço do usuário por meio de bibliotecas específicas. O sistema operacional não tem conhecimento direto dessas threads — ele enxerga apenas o processo como um todo.

Threads no espaço de Usuário

- Totalmente inseridos pelo usuário;
- Núcleo não tem informação sobre os threads;
- Contém uma tabela de threads;



Threads no espaço de Usuário

Vantagens

- As operações são mais rápidas a nível de usuário, se comparadas com o nível de kernel.
- Cada processo pode ter seu próprio algoritmo de escalonamento.
- Pode-se utilizar o recurso de threads, mesmo se o SO não o suportar.

Desvantagens

- O usuário precisa garantir o funcionamento do sistema.
- Pode ser bloqueante;

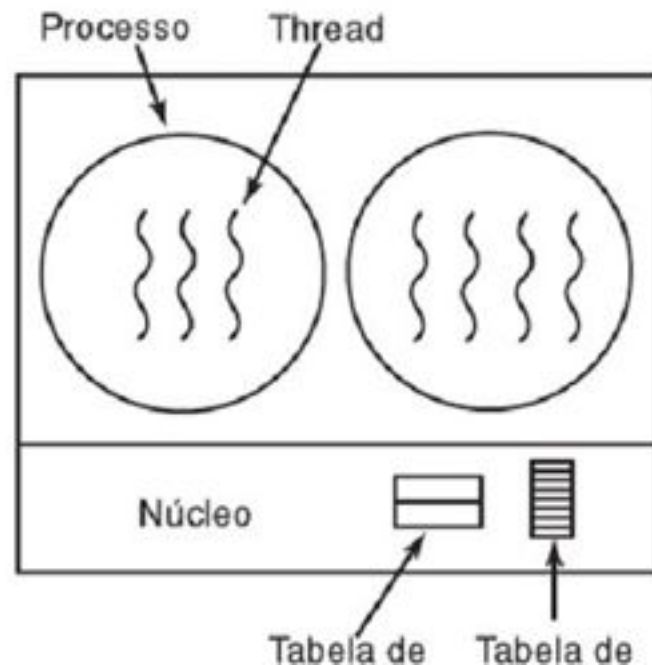
Threads no espaço do Núcleo

Threads de núcleo

São gerenciadas diretamente pelo sistema operacional. Nesse caso, cada thread é visível ao núcleo, que pode gerenciá-las.

Threads no espaço do Núcleo

- O núcleo tem uma tabela de threads, que acompanha todos os threads no sistema;
- Quando um thread quer criar um outro, ou mesmo destruir um thread já existente, ele faz uma chamada ao núcleo, que realiza a operação;



Threads no espaço do Núcleo

Vantagens

- Uma vez validado o algoritmo de escalonamento dentro do SO, todos os processos podem utilizá-lo.
- O tempo de ocupação na CPU fica melhor dividido se o kernel conhece todos os threads ativos.

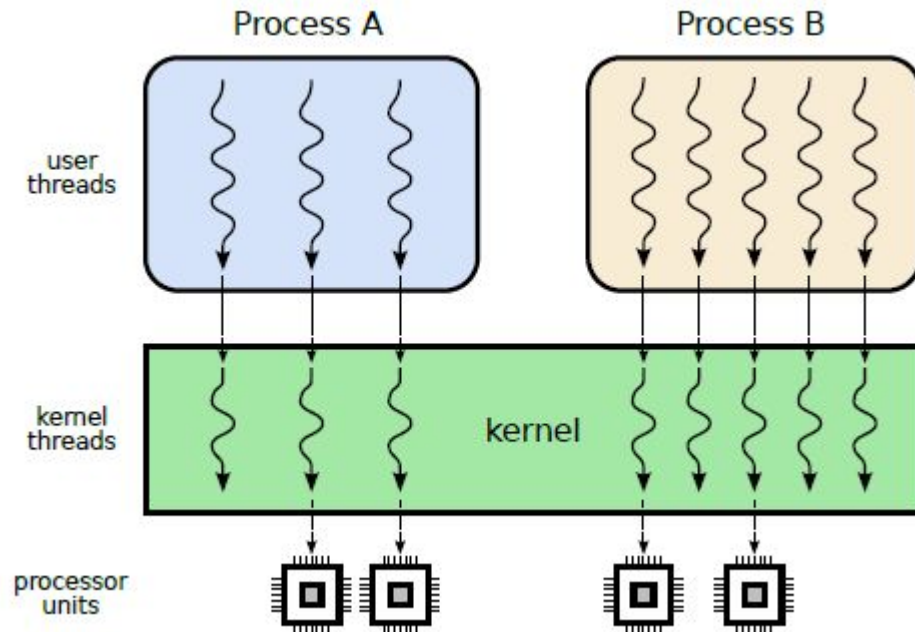
Desvantagens

- Operações são bem mais lentas a nível de kernel (políticas de proteção e segurança, diversas estruturas de kernel).
- Cada processo fica sujeito a política de escalonamento do próprio SO.

Modelo 1:1

- A necessidade de suportar aplicações multithread levou os desenvolvedores de sistemas operacionais a **incorporar a gerência dos threads dos processo no núcleo do sistema**. Para **cada thread de usuário** foi então **associado um thread correspondente dentro do núcleo (1:1)**, suprimindo com isso a necessidade de bibliotecas de threads.
- Este é o modelo mais frequente nos sistemas operacionais atuais, como o Windows NT e seus descendentes, além da maioria dos UNIXes. (Maziero, 2019)

Modelo 1:1



Modelo 1:1

Vantagens

- **Não Bloqueante:** caso uma thread de usuário solicite uma operação bloqueante, somente sua thread de núcleo correspondente será suspensa, sem afetar as demais threads do processo;
- **Justa:** distribuição de processamento entre as threads é mais justa;
- **Multinúcleo:** caso o hardware tenha mais de um processador, mais threads do mesmo processo podem executar ao mesmo tempo.

Desvantagem

- **Pouco escalável:** a criação de um número muito grande de threads impõe uma carga elevada ao núcleo do sistema, inviabilizando aplicações com muitas tarefas (como grandes servidores Web e simulações de grande porte, seriam melhores Thread pools).

Implementação de Thread (modo explícito)

Modo explícito

- No passado, cada sistema operacional definia sua própria interface para a criação de threads, o que levou a problemas de portabilidade das aplicações.
- Em 1995 foi definido o padrão IEEE POSIX 1003.1c, mais conhecido como **POSIX Threads** ou simplesmente **PThreads** [Nichols et al., 1996], que define uma interface padronizada para o uso de threads na linguagem C.
- Há outras bibliotecas de threads, como a do Windows e de Java, por exemplo.

Modo explícito

```
using System;
using System.Threading;

class Program {
    static void Trabalho() {
        for (int i = 0; i < 5; i++)
            Console.WriteLine("Rodando na thread " + Thread.CurrentThread.ManagedThreadId);
    }

    static void Main() {
        Thread t = new Thread(Trabalho);
        t.Start();
        t.Join(); // espera terminar
    }
}
```

Exemplo de código gerado em C#

Implementação de Thread (modo implícito)

Modo Implícito

Com o contínuo crescimento do processamento multicore, foram surgindo aplicações contendo centenas — ou até mesmo milhares — de threads. **Projetar essas aplicações não é tarefa simples.**

Uma forma de superar essas dificuldades e suportar melhor o projeto de aplicações com múltiplos threads **é transferir** a criação e o gerenciamento de threads dos desenvolvedores de aplicações para os compiladores e para as bibliotecas de tempo de execução.



Modo implícito

```
using System;
using System.Threading.Tasks;

class Program {
    static async Task Main() {
        Task t1 = Task.Run(() => Console.WriteLine("Tarefa 1"));
        Task t2 = Task.Run(() => Console.WriteLine("Tarefa 2"));

        await Task.WhenAll(t1, t2); // espera ambas terminarem
    }
}
```

Exemplo de código gerado em C#

Moral da história

Hoje não precisamos mais programar explicitamente com threads, mas precisamos entender os conceitos para saber como as ferramentas modernas funcionam, e assim aplicar as suas funcionalidades devidamente.

Qual a provável saída do código?

```
void* imprime(void* arg) {  
    char c = *(char*)arg;  
    for (int i = 0; i < 3; i++) printf("%c", c);  
    return NULL;  
}  
  
int main() {  
    pthread_t t1, t2;  
    char a = 'A', b = 'B';  
  
    pthread_create(&t1, NULL, imprime, &a);  
    pthread_create(&t2, NULL, imprime, &b);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
  
    printf("\n");  
    return 0;  
}
```

```

void* imprime(void* arg) {
    char c = *(char*)arg;
    for (int i = 0; i < 3; i++) printf("%c", c);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    char a = 'A', b = 'B';

    pthread_create(&t1, NULL, imprime, &a);
    pthread_create(&t2, NULL, imprime, &b);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n");
    return 0;
}

```

=

São respostas possíveis:

AABABB

ABBAAB

BBABAA

...

N respostas são possíveis

Na aula de Hoje, aprendemos:

- Conceito de Threads;
- As diferenças entre Threads e Processos;
- Os modelos de Threads de usuário e de Núcleo e suas relações;
- Modos de implementação de Threads (Explícito e implícito).

Referencial Bibliográfico

- Maziero, Carlos Alberto – ***Sistemas Operacionais: Conceitos e Mecanismos***
- Tanenbaum, Andrew S.; Bos, Herbert – **Sistemas Operacionais Modernos** (tradução de Modern Operating Systems, 4ª edição);
- Silberschatz, Abraham; Galvin, Peter Baer; Gagne, Greg – ***Fundamentos de Sistemas Operacionais*** (tradução de *Operating System Concepts*, 9ª edição)

SISTEMAS OPERACIONAIS

Exit